Smriti — Technical Documentation

Written against the code at commit `fa3e275` (working tree clean). Every behavioural claim below carries a `file:line` citation. Where the README and the code disagree, the code is described and the disagreement is flagged.

0. Read this first — where the README is out of date

The README predates four of the last six commits. Four gaps matter, and a reviewer who reads the README instead of the code will describe an app that no longer exists.

README / CLAUDE.md says	The code does	Where
"There is no ML and no cloud; it is a rules engine" (`README.md:75-76`); "NO LLM, NO cloud, NO ML model" (`CLAUDE.md`)	A logistic-regression classifier trained by SGD on-device sits alongside the rules engine and chooses the level	`js/core/model.js:1-150`, `js/core/store.js:399-447`

Module list omits `model.js`, `listen.js`, `demo-data.js` (`README.md:51`)	All three ship and are precached	`sw.js:19-21`
Speech input is not mentioned anywhere	Optional tap-to-talk speech recognition exists, and is the one feature that sends data off the device	`js/core/listen.js:1-204`
Voice is described only as Web Speech API	191 English and 189 Assamese lines are pre-rendered as MP3s and played as recordings; live synthesis is the fallback	`js/core/voice.js:125-190`, `tools/build-voice.py`
Docker not mentioned	Single-stage nginx image, port 8000	`Dockerfile`, `nginx.conf`, `docker-compose.yml`
Navigation map lists three exercises (`CLAUDE.md`)	Five, all reachable	`js/screens/patient/exercises.js:33`

"A wrong answer … gently shows the correct option, then advances" (`CLAUDE.md`)	Exercise 1 does exactly that; exercises 2–5 instead loop for a retry and never advance until the answer is right	`js/screens/patient/exercises.js:378-397` vs `js/screens/patient/exercises.js:549-558`

The `model.js`-versus-`CLAUDE.md` tension is the single most likely thing a technical judge will catch. It is answered in full at **Q1** and **Q2** in section 7. The short version: the standing rule forbids an LLM, a cloud call and a downloaded model file, and the code violates none of those — but "no ML model" is no longer a true description of what ships, and the honest word for `model.js` is *machine learning*.

---

1. What it is

Smriti is an offline-first **cognitive-engagement and memory-assistance** PWA for elderly people living with dementia in North-East India. It reads the day's reminders aloud, walks the patient through five practice activities that adapt to how the activities are going, shows named photographs of their people, and gives a caregiver a plain record of what happened. **It is not a diagnostic tool, not a medical device, and not treatment.** Nothing in it measures or infers a cognitive condition; every number it reports describes activity performance in the app and nothing else (`js/core/analytics.js:5-6`). The constraint set it was built under is: fully functional offline after one load; no build step; no framework; no login or accounts of any kind; usable on a ~1 GB Android phone with an old browser; and served over `http://` — `file://` breaks `fetch`, the manifest and the service worker.

What ships: **3,654 lines of JavaScript** across 27 ES modules, 1,370

lines of CSS, 1,582 lines of content JSON, 36 SVG illustrations, and 3.3 MB of pre-rendered English speech. No dependencies, at build time or at runtime.

---

2. Architecture — how a tap becomes a screen

Boot sequence

`js/app.js:22–58` is the whole of it, in order:

```
async function bootstrap() {
  // The store hydrates first: it holds the language the
  // chosen by, and every screen reads it synchronously a
  await initStore();

  if (isFirstRun()) {
    await importData(buildDemoData({ sample: true }));
  }

  const language = getSettings().language;
  const content = await loadContent(language);
  document.documentElement.lang = language;
  setVoiceLanguage(content.app.speechLang);
  await loadVoiceClips(language);
```

— `js/app.js:22–41`

Four awaits, then nothing else in the app ever awaits anything to render. The store finishes hydrating into an in-memory cache; a blank device seeds sample data; the content pack for the stored language is fetched; the voice layer is told which language tags to prefer and loads the index of pre-rendered clips. Only then are the ten routes registered (`js/app.js:43–52`), the router started (`js/app.js:54`), and the reminder

watcher armed (`js/app.js:57`). The service worker registers separately on `load`, so a failure there cannot delay the first paint (`js/app.js:62-66`).

Content injection

Every screen render function has the signature `(container, params, navigate, content)`. The router only knows about the first three, so `app.js` closes over the loaded pack once:

```
function withContent(renderFn, content) {
  return (container, params, navigate) => renderFn(contai
}
```

— `js/app.js:18-20`

That is the entire internationalisation mechanism at the code level. No screen imports a string; every screen is handed the pack.

The router

A hash router in 46 lines. `navigate(name, params)` stores the params in a module variable and sets `location.hash`; if the hash is already correct it re-renders directly (`js/core/router.js:11-19`). `renderCurrent()` looks the route up, tears down anything the previous screen left running, empties `#app`, and calls the render function (`js/core/router.js:21-37`).

The teardown is the non-obvious part:

```
  // A line belongs to the screen that started it. Leavin
  // here rather than in each screen's own exit handler —
  // that does not speak on entry (Patient Home, every ca
  // nothing to cancel the line still running underneath
  stopSpeaking();
  stopListening();
  app.innerHTML = '';
```

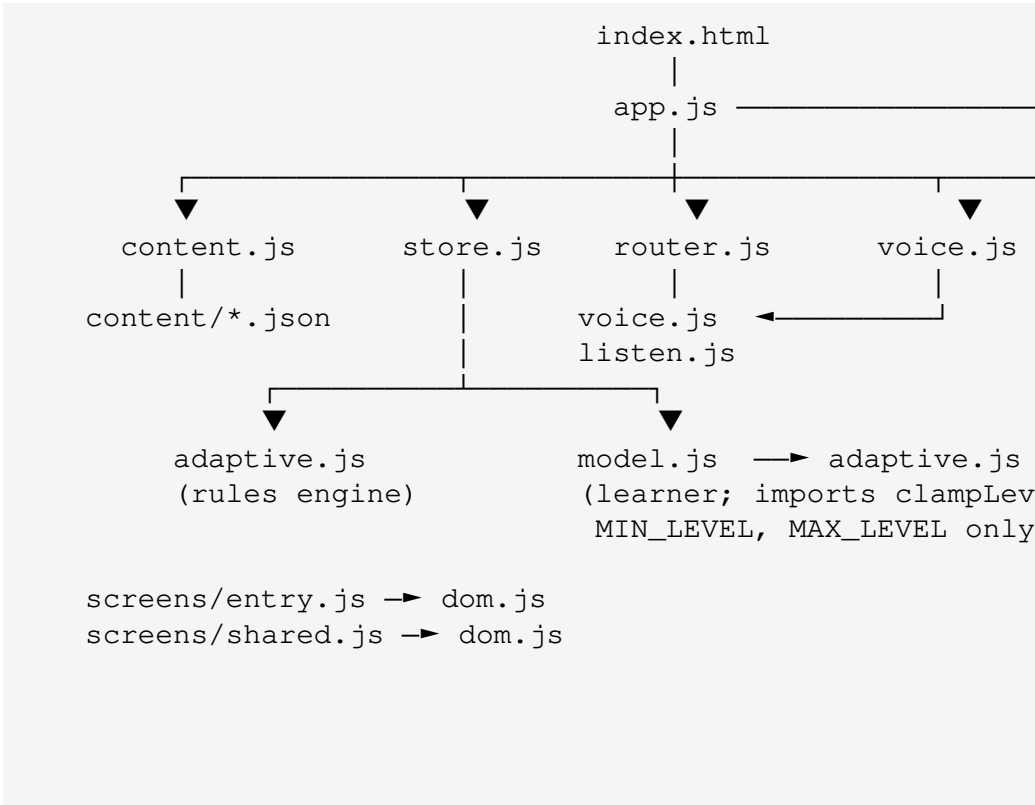
— `js/core/router.js:25-31`

The two worlds

There is no shared layout, no shared shell, and no shared navigation component between the patient and caregiver trees. They meet only at Entry (`js/screens/entry.js:11-15`) and at the two crossing points: the caregiver dashboard's "Switch to Patient" (`js/screens/caregiver/dashboard.js:22`) and the long-press corner on Patient Home (`js/screens/patient/home.js:37-45`). The patient tree has exactly one navigation control, the home icon (`js/screens/shared.js:5-14`); the caregiver tree has a menu and back buttons (`js/screens/shared.js:16-21`).

Module dependency graph

Arrows point from dependent to dependency.



```
screens/patient/home.js ———► dom, shared, voice, for
screens/patient/today.js ———► dom, shared, voice, for
screens/patient/memories.js —► dom, shared, voice, sto
screens/patient/exercises.js ► dom, shared, voice, lis
screens/patient/reminder-card.js ► dom, voice, store

screens/caregiver/dashboard.js —► dom, alerts
screens/caregiver/alerts.js ———► dom, analytics, stor
screens/caregiver/overview.js —► dom, shared, alerts,
screens/caregiver/reminders.js —► dom, shared, format,
screens/caregiver/engagement.js ► dom, shared, format,
screens/caregiver/memories-settings.js ► dom, shared,
```

Three properties fall out of that graph and are worth stating because they are what make the codebase auditable:

1. **`adaptive.js` and `analytics.js` are pure.** Neither imports the store. Data is passed in; shapes come out (`js/core/adaptive.js:1-3`, `js/core/analytics.js:1-6`).
2. **`model.js` depends on `adaptive.js`, never the reverse.** The rules engine cannot be influenced by the learner; the learner is bounded by the rules (`js/core/model.js:19`).
3. **Only `store.js` writes to disk.** Every screen reads through it synchronously.

Why the store hydrates synchronously

```
// Reads are synchronous against an in-memory cache hydra
// so screens can render without awaiting. Writes update
// and persist in the background through a serialised que
```

— `js/core/store.js:7-9`

IndexedDB's API is callback-and-event based; awaiting it inside a render function would mean every patient screen has a frame where it is empty. On a 1 GB phone that reads as a stutter or a crash. So `initStore()` pays the async cost once (`js/core/store.js:110-139`) and everything after it is a synchronous read against `cache`.

---

3. Feature by feature

Offline shell and service-worker caching

What it does — On first load the service worker copies the entire app — markup, styles, all 27 scripts, both content packs, every illustration and icon, and every pre-rendered English speech clip — into a named cache. After that the app opens and runs with the network switched off, indefinitely.

Where it lives — `sw.js:116-124`:

```
self.addEventListener('install', (event) => {
  event.waitUntil(
    caches.open(CACHE_NAME).then(async (cache) => {
      await cache.addAll(APP_SHELL);
      await precacheAudio(cache);
    })
  );
  self.skipWaiting();
});
```

and `sw.js:135-151`:

```
self.addEventListener('fetch', (event) => {
  event.respondWith(
    caches.match(event.request).then((cached) => {
      if (cached) return cached;
      return fetch(event.request).then((response) => {
        if (response.ok && event.request.url.indexOf('/as
          const copy = response.clone();
          caches.open(CACHE_NAME).then((cache) => cache.p
        }
        return response;
      });
```

```
    })
  );
});
```

How it works — `APP_SHELL` (`sw.js:3-80`) is an explicit 79-entry list; there is no glob and no manifest generator, because there is no build step. Audio is handled separately: the list of clips is read at install time from `assets/audio/en/index.json` rather than hard-coded, so re-running the voice build cannot leave `sw.js` stale (`sw.js:82-90`). Serving is cache-first — `caches.match` wins over the network for everything — and a miss under `/assets/audio/` is written back into the cache on the way through (`sw.js:143-146`). `activate` deletes every cache whose name is not the current `CACHE_NAME` (`sw.js:126-133`), which is why `CACHE_NAME` must be bumped on every deploy; it currently reads `smriti-shell-v12` (`sw.js:1`).

Why it was built this way — Clip-by-clip precaching rather than one `addAll`:

```
      // One at a time with its own catch: addAll() throw
      // if a single file is missing, and silence is wors
      for (const file of files) {
        await cache.add(file).catch(() => {});
      }
```

— `sw.js:105-109`

And only English is precached:

```
// Only English is precached up front. The Assamese set i
// and a device that never switches language should not p
// connection — the fetch handler caches those clips the
// and until then that language simply speaks live.
```

— `sw.js:86-89`

The measured cost of that decision: the app shell is **261 KB**; the English audio adds **3.26 MB**, for **3.51 MB** at install. Precaching Assamese as well would roughly double it, on connections where that is the difference

between installing and giving up.

Known limits — (a) A device that switches to Assamese must be online once for each Assamese clip, or that language speaks with the device engine until it has been. (b) `manifest.json:12` references `assets/icons/icon-maskable-512.png`, which is **not** in `APP_SHELL` (`sw.js:38-43`) — an offline install can miss the maskable icon. (c) Cache-first means the nginx `no-cache` headers on `index.html` and `/content/` (`nginx.conf:19-27`) have no effect once the worker is installed; only a `CACHE_NAME` bump ships a new build. (d) A cache miss with no network rejects rather than returning a friendly offline page — acceptable only because everything the app requests is precached.

Persistence: IndexedDB with a localStorage mirror

What it does — Stores five collections — reminders, memories, engagement, settings, activity — on the device. Nothing is ever sent anywhere.

Where it lives — `js/core/store.js:110-139`:

```
export async function initStore() {
  if (ready) return ready;
  ready = (async () => {
    cache = defaults();
    try {
      db = await openDB();
    } catch (err) {
      db = null;
    }
    for (const key of COLLECTIONS) {
      let stored;
      if (db) {
        try { stored = await idbRead(key); } catch (err)
      }
      if (stored === undefined) stored = lsRead(key);
      cache[key] = hydrateCollection(key, stored);
    }
```

and the write path, `js/core/store.js:165-177`:

```
function persist(key) {
  const value = cache[key];
  lsWrite(key, value);
  writeQueue = writeQueue.then(async () => {
    if (!db) return;
    try { await idbWrite(key, value); } catch (err) { /*
  });
  return writeQueue;
}
```

How it works — Every write updates the in-memory cache first, then writes the localStorage mirror **synchronously**, then appends the IndexedDB write to a serialised promise chain. Reads never touch disk. On boot, IndexedDB is preferred and localStorage fills any key it does not have. `hydrateCollection` (`js/core/store.js:103-108`) merges a stored object over the current defaults, so a payload saved before a field existed still gets that field — which is why `speechInput` defaults correctly on a phone that has been in use since before the feature landed (`js/core/store.js:132-135`).

Why it was built this way —

```
// Local-only persistence. IndexedDB is the real store (p
// run to megabytes, well past the localStorage cap); loc
// fallback for old Android browsers where IndexedDB is m
```

— `js/core/store.js:1-3`

Photos are stored inline as data URLs. localStorage alone caps out around 5 MB; IndexedDB alone is unavailable or blocked on some old and privacy-hardened Android browsers. The mirror gets both.

Known limits — The mirror can silently fail on quota (`js/core/store.js:93-99` swallows the exception and returns `false`, and nothing reads that return value). On a device with no IndexedDB *and* a full localStorage, writes are lost with no user-visible warning. Round history is capped at 60 per exercise and sessions at 30 (`js/core/`

`store.js:21-22, 426-428, 456-458`) — long-term history is deliberately not kept.

Routing and the two-world separation

What it does — Ten routes, one hash each, two disjoint screen trees. The caregiver side is reachable from Patient Home only by holding a dim corner for two seconds.

Where it lives — `js/screens/patient/home.js:35-45`:

```
// A UX gate, not authentication: dim, low contrast, an
// so a confused patient cannot stumble into the caregi
const corner = el('div', {
  class: 'caregiver-corner',
  onpointerdown: () => {
    pressTimer = setTimeout(() => navigate('caregiver-d
  },
  onpointerup: () => clearTimeout(pressTimer),
  onpointercancel: () => clearTimeout(pressTimer),
  onpointerleave: () => clearTimeout(pressTimer),
});
```

`LONG_PRESS_MS` is 2000 (`js/screens/patient/home.js:8`). The corner is a 44 px circle at 5 % opacity (`css/patient.css:165-172`).

How it works — Pointer-down starts a timer; any of up, cancel or leave clears it. Two seconds of contact navigates. The same destination is also one plain tap from the Entry screen (`js/screens/entry.js:12-15`), which is the route a caregiver actually uses.

Why it was built this way — The failure mode is a disoriented patient tapping into a screen full of data density and their own performance history. A 2-second hold is long enough that it cannot happen by accident, and short enough that a caregiver does not have to be taught it.

Known limits — This is **not** a security boundary and is documented as one (`README.md:104-106`). Anyone holding the phone for two

seconds is in. There is no lock, no PIN and no encryption; see section 5.

A second limit, of the routing itself: route params live in a module variable, not the URL (`js/core/router.js:5, 12`). A page reload or an Android hardware-back press re-renders the route with whatever params were set last, not the params that belonged to that history entry. Within the patient rails this is invisible — successive photos never change the hash — but it means no screen is addressable or restorable by URL.

Voice output: recordings first, live synthesis second

What it does — Every fixed patient-facing line is played back as an MP3 recorded once with a neural voice. Lines assembled at runtime from a name or a time fall through to the device's own `speechSynthesis`. Nothing in the app ever waits on either.

Where it lives — `js/core/voice.js:125-142`:

```
export function speak(text, onEnd) {
  if (!text) return;
  stopSpeaking();
  const gen = generation;
  const line = String(text).trim();
  const entry = clips ? clips[line] : null;
  const file = entry ? entry.f : null;

  // Chrome drops an utterance queued in the same tick as
  // over on the next one. Recordings go through the same
  // pacing identical whichever source ends up speaking.
  pending = setTimeout(() => {
    pending = null;
    if (gen !== generation) return;
    if (file && playClip(file, line, onEnd, gen)) return;
    speakLive(line, onEnd, gen);
  }, 60);
}
```

How it works — `tools/build-voice.py` walks the `entry` and `patient` sections of each content pack, collects every string that is longer than one character, contains a letter, and has no `{placeholder}` (`tools/build-voice.py:55-83`), renders each with `edge-tts` at –10 % rate (`tools/build-voice.py:52, 108-111`), names the file by a SHA-1 prefix of the text (`tools/build-voice.py:86-87`), measures its real duration with `ffprobe` (`tools/build-voice.py:90-105`), and writes `index.json` mapping the exact line to `{f, ms}`. At runtime `loadVoiceClips()` fetches that index (`js/core/voice.js:49-64`) and `speak()` looks the line up verbatim — no hashing in the browser, deliberately:

```
    # voice.js looks lines up by their exact text, so no
    # the browser — SubtleCrypto is unavailable over plai
    # address, which is exactly how the demo phone will l
```

— `tools/build-voice.py:156-158`

The measured durations feed back into the screens' advance timers (`js/core/voice.js:120-123`, used at `js/screens/patient/exercises.js:297-300`), so a recording is never cut off by a fixed ceiling.

The **generation counter** is the piece that took a commit of its own to get right:

```
// Bumped by every speak() and every stopSpeaking(). A li
// current when it starts and checks it before acting on
// arrives later. This is what keeps a cancelled line qui
//
// Both engines report a line we deliberately stopped as
// to it: pausing an <audio> whose play() has not resolve
// promise with AbortError, and speechSynthesis.cancel()
// utterance it just killed. Without this counter the fir
// from a broken file — so the line falls back to the liv
// itself again over the top of whatever started next — a
// the caller a line finished when it never did.
let generation = 0;
```

— `js/core/voice.js:21-32`

The two consequences are at `js/core/voice.js:154-176` (a stopped clip must not be re-spoken live, and must not report `onEnd`) and `js/core/voice.js:207-219` (a cancelled utterance's `onend` must not be mistaken for completion).

Voice selection and the Assamese chain — Each pack declares an ordered list of language tags. English is `["en-IN","en-GB","en-US","en"]` (`content/en.json:8-13`); Assamese is `["as-IN","as","bn-IN","bn"]` (`content/as.json:8-13`). `pickVoice()` takes the first installed voice whose tag prefix-matches, in order (`js/core/voice.js:87-91`), and stays unresolved while Chrome's voice list is still empty (`js/core/voice.js:85-86`), re-resolving on `voiceschanged` (`js/core/voice.js:253-257`). The same reasoning drives the recording build: `'as': 'bn-IN-TanishaaNeural'` (`tools/build-voice.py:41-48`) — Assamese neural voices do not exist, and Assamese is written in the Bengali-Assamese script, so a Bengali voice pronounces it far closer than any Latin-script voice.

Why it was built this way —

```
The app leads with its voice, and speechSynthesis on a lo
like a robot reading a form. ... At runtime the app plays
fetched, nothing is synthesised, and the voice is identic
which also removes the demo-day risk of landing on a phon
missing or slow to wake.
```

— `tools/build-voice.py:4-11`

Known limits — Eight lines in the English pack contain `{placeholders}` and are therefore **never** recorded: the reminder-card announcement, the Today "next reminder" sentence, Exercise 3's `"Tap the {object}."`, Exercise 4's `"{name} — let us put these in order."`, Exercise 1's photo question, and the home strip and photo counter (which are not spoken). Photo *names*, spoken on the Memories screen (`js/screens/patient/memories.js:45`), have no recording either. On a phone with **no** TTS engine at all, those lines are silent — Exercise 3 and Exercise 4 prompt only in text. And

`voiceStatus()` reports `available: true` whenever recordings exist (`js/core/voice.js:106-114`), so Caregiver Settings will say "Speaking with the 191 lines recorded into the app" on a device that cannot in fact speak those eight.

Optional speech input (STT)

What it does — In Exercise 1 the patient may press a microphone button and say the answer instead of tapping it. Tapping is unchanged and always works.

Where it lives — `js/screens/patient/exercises.js:216-295` builds the button; the match feeds the ordinary answer path:

```
      // Speech input, where the device and the caregiver b
      // choose() — the same path a tap takes — so the lear
      // and the wrong-answer handling cannot tell the two
      // on the advance path: if it never fires, the round
      const mic = speechInput(options, (id) => choose(id),
```

— `js/screens/patient/exercises.js:399-403`

The matching rule is in `js/core/listen.js:187-204`:

```
export function matchSpoken(heard, options) {
  const transcripts = [].concat(heard || []).filter(Boole
  if (!transcripts.length || !options || !options.length)

  const scored = options.map((option) => {
    let best = 0;
    for (const transcript of transcripts) {
      best = Math.max(best, score(transcript, option.labe
    }
    return { id: option.id, score: best };
  }).sort((a, b) => b.score - a.score);

  const winner = scored[0];
  const runnerUp = scored[1];
```

```
  if (!winner || winner.score < MIN_SCORE) return null;
  if (runnerUp && winner.score - runnerUp.score < MIN_MAR
  return winner.id;
}
```

How it works — A tap silences the app's own question, waits 180 ms (`js/screens/patient/exercises.js:28-31`) so the recogniser does not hear the tail of it, then opens the microphone for one utterance with up to three alternatives (`js/core/listen.js:64-67`). Each alternative is scored against each on-screen label: exact match 1.0; every word of the label present in the transcript 0.95 (so "I think that's the gamosa" resolves); otherwise the best of whole-string and per-word normalised Levenshtein similarity (`js/core/listen.js:162-182`). A winner must clear **0.7 absolute** and lead the runner-up by **0.12** (`js/core/listen.js:122-123`), or nothing happens at all.

Why it was built this way — Two comments carry the design:

```
    Deliberately conservative. Resolving the WRONG option
    transcript was garbled would invent a mistake the pati
    and a false wrong answer is far worse here than hearin
    all: it trains the learner on a lie and it tells someo
    loss they were wrong when they were not.
```

— `js/core/listen.js:114-119`

```
// Honest note on offline: the browser's recogniser is cl
// Chrome — it streams audio to a server. There is no off
// recognition available to a web page.
```

— `js/core/listen.js:11-14`

A `network` or `service-not-allowed` error retires the microphone for the session rather than leaving a dead button on screen (`js/core/listen.js:77-85`), and the button is not rendered at all where speech cannot work or the caregiver has switched it off (`js/screens/patient/exercises.js:217`). A transcript that matches nothing is never recorded as a wrong answer; the patient is told plainly and the round waits (`js/screens/patient/exercises.js:273-286`).

Known limits — (a) It needs a network. This is the one part of the app that breaks in airplane mode, and the one part that leaves the device. It is disclosed in the caregiver UI verbatim (`content/en.json:733`). (b) It is wired into **Exercise 1 only** — exercises 2–5 have no microphone. (c) Matching is edit-distance over labels, with no acoustic model of Assamese; the thresholds were tuned against English labels.

The five exercises

All five share one session controller. `SEQUENCE` (`js/screens/patient/exercises.js:33`) is `['exercise-1' … 'exercise-5']`; `stages` wraps it in an intro and a praise screen (`js/screens/patient/exercises.js:67`); `paint()` dispatches (`js/screens/patient/exercises.js:142-159`) and, critically, silences whatever the outgoing screen was doing before drawing the incoming one:

```
  function paint() {
    clearTimers();
    // Nothing from the screen being replaced may still b
    // once the next one is up. Leaving this to the incom
    // speak() meant any stage that does not speak immedi
    // let the previous line run on underneath it.
    stopSpeaking();
    stopListening();
```

— `js/screens/patient/exercises.js:142-149`

A session-scoped token guards every timer, so a timer from an abandoned session cannot fire into a screen that no longer exists (`js/screens/patient/exercises.js:36-37, 77-89, 104-111`).

Difficulty enters each exercise through one lever, re-read every round from the store so a level change between rounds takes effect immediately (`js/screens/patient/exercises.js:54-59`).

#	Name	Mechanic	Difficulty lever (levels 1/2/3)	Wro answ

1	Remembering	A scene, then "which one do we drink tea from?"	option count **2** / **3** / **4** (`content/en.json`, `reminiscence.optionsByLevel`)	reveal the rig one, th advan
2	Matching	Every picture face-up twice; find one pair	pair count **2** / **3** / **4** (`matching.pairsByLevel`)	gentle line, ti reset, again
3	Find It	"Tap the gamosa"	option count **3** / **4** / **5** (`spot.optionsByLevel`)	gentle line, tr again
4	Daily Routine	Put a routine in order	step count **3** / **4** / **5** (`routine.stepsByLevel`)	"that c comes later", nothin moves
5	What Comes Next	ABAB… / AABAAB / ABCABC, fill the blank	template **and** option count **2** / **3** / **4** (`pattern.templatesByLevel`, `pattern.optionsByLevel`)	gentle line, tr again

Exercise 1 — Remembering. `js/screens/patient/exercises.js:367-431`. The pool is the 12 authored items plus, if the caregiver has saved two or more named photos, one "Which one is *name*?" round per photo (`js/screens/patient/exercises.js:318-353`). Photo rounds render image-only, because "Printing the names under the photos would give the answer away" (`js/screens/patient/exercises.js:347-348`). The wrong-answer path:

```
    function choose(chosenId) {
```

```
      // One resolved answer per round: later taps do not
      if (resolved) return;
      resolved = true;
      stopListening();
      const correct = chosenId === item.answer;

      for (const button of buttons) {
        button.disabled = true;
        if (button.dataset.optionId === item.answer) butt
        else if (button.dataset.optionId === chosenId) bu
      }
```

— `js/screens/patient/exercises.js:378-389`

Every button is disabled on the first tap, the correct one is highlighted, and the round is recorded and advanced with a **3800 ms** reveal window rather than the 2400 ms used for a correct answer (`js/screens/patient/exercises.js:22-23, 391-396`). This is the behaviour `CLAUDE.md` specifies.

Exercises 2, 3 and 5 take a different and, I would argue, better line: a wrong tap is marked, a gentle "Not quite — try again" is spoken, the marking is cleared, the prompt is re-spoken, and the round stays open until the patient gets it (`js/screens/patient/exercises.js:549-558` for Exercise 3; 484-492 and 718-726 for 2 and 5). The `busy` flag holds input during the pause so repeated taps do nothing (`js/screens/patient/exercises.js:534, 718`). The round is still recorded as incorrect for the adaptive engines, via `correct: attempts === 1` (`js/screens/patient/exercises.js:544`). **Exercise 4** does the same and records `correct: attempts === 0` (`js/screens/patient/exercises.js:625`).

Why — There is no fail state anywhere: `js/screens/patient/exercises.js:1-3` states the rule, and the only reason Exercise 1 reveals-and-advances rather than looping is that a "which one is Ama?" round has no second guess worth making.

Known limits — Exercise 1 always records `attempts: 1` (`js/screens/patient/exercises.js:392`), so for that exercise the

learner's `recentRetries` feature is pinned at −1 and carries no information — it acts as a second bias term. Exercise 4's item pool is only 4 routines (`content/en.json`, `routine.items`) and rounds cycle with `%`, so a long session repeats. Exercise 5 silently skips itself if the object pool is smaller than the template needs (`js/screens/patient/exercises.js:676`) — unreachable with the shipped pool of 20, but it is a silent skip rather than a graceful fallback.

Adaptive difficulty: the rules engine

What it does — Three correct in a row raises the level; two wrong lowers it; at the easiest level it repeats instead of dropping further.

Where it lives — `js/core/adaptive.js:26-54`:

```
export function nextState(state, correct) {
  let difficulty = clampLevel(state.difficulty);
  let correctStreak = state.correctStreak || 0;
  let wrongStreak = state.wrongStreak || 0;
  let lastChange = null;

  if (correct) {
    correctStreak += 1;
    wrongStreak = 0;
    if (correctStreak >= RAISE_AFTER_CORRECT) {
      correctStreak = 0;
      if (difficulty < MAX_LEVEL) { difficulty += 1; last
    }
  } else {
    wrongStreak += 1;
    correctStreak = 0;
    if (wrongStreak >= LOWER_AFTER_WRONG) {
      wrongStreak = 0;
      // At the easiest level there is nowhere to go down
      lastChange = difficulty > MIN_LEVEL ? 'down' : 'rep
      if (difficulty > MIN_LEVEL) difficulty -= 1;
    }
  }
  return { difficulty, correctStreak, wrongStreak, lastCh
```

```
}
```

How it works — Pure function, no I/O, three levels (`js/core/adaptive.js:5-8`). Each exercise carries its own state; the caregiver's baseline seeds it (`js/core/store.js:368-380`) and re-applying the baseline resets every exercise's level and streaks while keeping the history (`js/core/store.js:343-354`).

Why it was built this way — It is the auditable floor. A caregiver, a clinician or a judge can read 28 lines and know exactly what the app will do. The caregiver screen describes it in those terms, in the pack, not in code: "3 correct in a row moves up, 2 incorrect eases back. It responds to activity performance only." (`content/en.json:695`).

Known limits — Three levels is coarse. Streak-based adaptation is memoryless across sessions in the sense that it does not weigh recency or difficulty of the specific item — only the run of outcomes.

The on-device learner

What it does — Predicts how likely this patient is to answer correctly at a given level, then proposes the level whose predicted success sits nearest 75 %. It may move the level by **one step at most** from wherever the rules engine landed, and only when that is clearly better.

Where it lives — `js/core/model.js:120-150`:

```
export function proposeLevel(model, rounds, rulesLevel) {
  const current = safeModel(model);
  const base = clampLevel(rulesLevel);
  const predicted = predict(current, base, rounds);

  if (current.trained < MIN_ROUNDS_TO_STEER) {
    return { level: base, predicted, steering: false,
             roundsUntilSteering: MIN_ROUNDS_TO_STEER - c
  }

  let level = base;
  let best = Math.abs(predicted - TARGET_SUCCESS);
```

```
  let chosen = predicted;

  for (const candidate of [base - 1, base + 1]) {
    if (candidate < MIN_LEVEL || candidate > MAX_LEVEL) c
    const p = predict(current, candidate, rounds);
    const gap = Math.abs(p - TARGET_SUCCESS);
    if (gap < best - MIN_GAIN || (gap < best && candidate
      level = candidate; best = gap; chosen = p;
    }
  }
  return { level, predicted: chosen, steering: true, roun
}
```

How it works, in order of execution. On every answered round, `store.recordRound` (`js/core/store.js:399-447`) does five things in this order:

1. `predict()` the outcome *before* seeing it, and keep that number on the round so the caregiver screen can plot expectation against outcome (`js/core/store.js:404-406`).
2. `learn()` — one SGD step, trained on the level the round was actually played at and the history as it stood *before* this answer (`js/core/store.js:408-410`).
3. Run the rules engine (`js/core/store.js:412-415`).
4. `proposeLevel()` inside the rules' ±1 bound (`js/core/store.js:430-435`).
5. Recompute `lastChange` against the level the patient will *actually* meet, so the spoken level-up line is never said for a move that did not happen (`js/core/store.js:437-442`).

Features — four, all scaled to roughly –1…1, all knowable before the round is answered (`js/core/model.js:75-87`):

```
export function features(level, rounds) {
  const recent = (rounds || []).slice(-WINDOW);
  const accuracy = mean(recent.map((round) => (round.corr
  const attempts = mean(recent.map((round) => Math.min(3,
  return [
    1,
```

```
    clampLevel(level) - 2,                                    //
    (accuracy - 0.5) * 2,                                     //
    Math.max(-1, Math.min(1, (attempts - 1.5) * 2)),          //
  ];
}
```

`bias`, `level`, `recentAccuracy` over an 8-round window, `recentRetries` over the same window. Learning rate 0.12, weight decay 0.002 applied to every weight except the bias, which "carries the patient's overall success rate and should be free" (`js/core/model.js:106-112`). The prior is `INITIAL_WEIGHTS = [1.1, -0.5, 0, 0]` (`js/core/model.js:43`) — at the middle level, about 75 % correct, with higher levels harder.

Guardrails — three, all constants at the top of the file (`js/core/model.js:23-36`): it does not steer at all until it has seen `MIN_ROUNDS_TO_STEER = 6` rounds; a candidate level must beat the rules' choice by `MIN_GAIN = 0.03` in distance-to-target, or ties fall to the easier level; and the candidate set is only `[base - 1, base + 1]`.

Why it was built this way —

```
// Online logistic regression, trained by stochastic grad
// device, from this one patient's own rounds. There is n
// model file to download, no network call: it starts fro
// learns as the person practises. Everything here is ~10
// which is the point — a clinician can audit every decis
//
// Claim discipline: this predicts SUCCESS IN AN ACTIVITY
// not a measure of the person, and it infers no cognitiv
```

— `js/core/model.js:6-13`

Does it help? Measured, not asserted. `tools/model-harness.py` mirrors the same arithmetic in Python — reading every constant straight out of `model.js` and `adaptive.js` so the mirror cannot drift (`tools/model-harness.py:32-53`) — and drives four synthetic patients whose true per-level success rates are known, 400 rounds × 40 seeds

each. Current output:

```
patient                                best   learner     ru
-------------------------------------------------------------
steady (best at level 2)              74.0%     75.6%     62
strong (best at level 3)              76.0%     79.9%     77
struggling (best at level 1)          72.0%     70.7%     56
declining over time                   60.3%     58.0%     52
```

`best` is the rate the single best available level would have given that patient (`tools/model-harness.py:166-172`); `learner` and `rules` are the realised success rates after a 40-round warm-up. The learner is **12.7, 2.2, 14.5 and 5.6 points** better than the rules alone, and within 5 points of the best reachable rate for all four. Level changes on 2–8 % of rounds, so it does not oscillate. Mean calibration error — |predicted – true probability| — is 0.046 to 0.077. (`learner` exceeding `best` for two patients is not magic: `best` is the rate of one fixed level, and mixing levels can beat it.)

Known limits — (a) The harness validates the control loop against synthetic patients drawn from a Bernoulli-per-level model. It proves the loop converges, tracks a drifting patient and does not thrash. It proves **nothing** about real people. (b) With three levels the target is often unreachable exactly. (c) The model is per-exercise and per-device; it starts cold on a new phone unless a backup is restored. (d) For Exercise 1 the `recentRetries` feature is constant (see above). (e) It predicts success in an activity. It is not a measure of the person, and no screen presents it as one.

Reminders and the watcher

What it does — Four reminder types with a time and an optional note. When one comes due, a full-screen card appears with a single Done button.

Where it lives — `js/screens/patient/reminder-card.js:60-74`:

```
function tick(content, navigate) {
  if (INTERRUPTIBLE.indexOf(currentRoute()) === -1) retur
  // Never stack a second card over one still waiting to
  if (document.querySelector('.reminder-overlay')) return
  const due = getDueReminders();
  if (!due.length) return;
  const app = document.getElementById('app');
  if (app) app.appendChild(buildDueCard(due[0], content,
}

export function startReminderWatch(content, navigate) {
  if (timer) clearInterval(timer);
  timer = setInterval(() => tick(content, navigate), CHEC
  return timer;
}
```

How it works — A 30-second poll (`js/screens/patient/reminder-card.js:14`) started once at boot (`js/app.js:57`). `getDueReminders` returns every reminder whose time has passed today and which is not marked done (`js/core/store.js:275-281`); `markReminderDone` stamps `lastDoneAt` and appends to a capped log (`js/core/store.js:283-293`). Done also returns the patient to Home (`js/screens/patient/reminder-card.js:36-41`).

Why it was built this way —

```
// Checking only when Patient Home is drawn meant a remin
// while the patient sat looking at the screen and nothin
// polls, so the card appears on its own.
//
// It cannot fire while the app is closed — that needs an
// is not available to a page without a server. Documente
```

— `js/screens/patient/reminder-card.js:3-8`

and the interruption rule:

```
// Routes where a reminder may interrupt. An exercise in
// alone: the card would break the round, and the patient
```

— `js/screens/patient/reminder-card.js:16-18`

Known limits — It fires only while the app is open and in the foreground. There is no background alarm, no notification and no wake-up: doing that from a web page requires a push server, which the no-network constraint forbids. This is the single largest functional gap in the product, and it is stated in the README (`README.md:80-82`) rather than glossed.

Caregiver Overview, Engagement, Reminders manager, Memories & Settings

Overview (`js/screens/caregiver/overview.js:14-46`) is six stats plus an alert list, ending in the required disclaimer (`js/screens/caregiver/overview.js:43`, text at `content/en.json:640`). Alerts are computed once and shared with the dashboard badge so the two can never disagree (`js/screens/caregiver/alerts.js:1-2, 14-41`). The alert rules — `QUIET_DAYS = 3`, `STRUGGLE_ROUNDS = 6`, `STRUGGLE_RATE = 0.4` — are in `js/core/analytics.js:102-104`, and the file states its own claim discipline:

```
    Alerts. Descriptive only: each one states something th
    never what it might mean about the person.
```

— `js/core/analytics.js:97-100`

Engagement (`js/screens/caregiver/engagement.js:20-66`) is where the adaptation is shown rather than asserted: a 14-day practice bar chart, and per exercise a level pill, first-try count, a first-try accuracy sparkline with the model's expectation drawn over it as a dashed line, a level sparkline, and the last ten round outcomes.

```
        // The dashed overlay is what the model expected be
        // answered. Tracking the solid line is the learner
        // away from it is the learner being wrong, and bot
```

— `js/screens/caregiver/engagement.js:104-106`

The learner reports its own state honestly, including when it has not started: `modelNote` prints "Watching for now — it starts adjusting after *n* more rounds" until `trained >= MIN_ROUNDS_TO_STEER` (`js/screens/caregiver/engagement.js:134-148`, strings at `content/en.json:704-710`).

Reminders manager (`js/screens/caregiver/reminders.js:13-133`) is a list, an add form and an edit form, keyed on `params.form` being `true` or an id (`js/screens/caregiver/reminders.js:16-21`). Time is the only required field (`js/screens/caregiver/reminders.js:89-100`); delete confirms (`js/screens/caregiver/reminders.js:102-106`).

Memories & Settings (`js/screens/caregiver/memories-settings.js:48-77`) carries photo add/delete, the language select, a voice-status readout, the baseline difficulty toggle, the speech-input switch, backup/restore, and the sample-data banner. Photos are downscaled to a 640 px long edge at JPEG 0.75 before storage, because "a phone camera JPEG would otherwise be several megabytes per photo" (`js/screens/caregiver/memories-settings.js:16-46`).

Backup/restore (`js/screens/caregiver/memories-settings.js:250-310`) writes the whole store to a JSON file with an `app: 'smriti'` marker and refuses anything that does not carry it (`js/screens/caregiver/memories-settings.js:256-290`); restore confirms, replaces every collection, and reloads.

```
// Backup is the honest version of "sync for remote areas
// controls, moved by hand, with no server and no account
```

— `js/screens/caregiver/memories-settings.js:247-249`

Known limits — The backup file is plain, unencrypted JSON containing photos and history. `clearAll()` stamps `lastActiveAt = Date.now()` (`js/core/store.js:484-486`) to stop the sample data reseeding, so a freshly wiped device reports "Last active: Just now" on Overview — cosmetically wrong. Charts are `role="img"` with an `aria-label` (`js/core/chart.js:34-35, 92-93`) but expose no underlying values to a screen reader.

Internationalisation

What it does — Every user-visible string, every image path, and the ordered TTS language chain live in `content/<code>.json`. Adding a language is a file plus one line in that file.

Where it lives — `js/core/content.js:8-17`:

```
// Content packs are plain JSON. Adding a language means
// content/<code>.json and listing it under app.languages
export async function loadContent(language) {
  if (content) return content;
  const code = language || 'en';
  content = await fetchPack(code);
  if (!content && code !== 'en') content = await fetchPac
  if (!content) throw new Error('no content pack could be
  return content;
}
```

The pack base is resolved against the module's own URL, not the page's (`js/core/content.js:3-6`) — which is what lets the tools pages in `tools/` load the same packs.

How a third language is added, end to end.

1. Copy `content/en.json` to `content/ne.json` and translate the values.
2. Add `"ne": "नेपाली"` to `app.languages` **in every pack** (`content/en.json:4-7`) — that object is what populates the language select (`js/screens/caregiver/memories-settings.js:145-149`).
3. Set `app.speechLang` to the ordered fallback chain for that language (`content/as.json:8-13` is the worked example).
4. Add `'ne': '<voice-name>'` to `VOICES` in `tools/build-voice.py:45-48` and run it to generate the recordings — optional; without it the language speaks live.
5. Add `'./content/ne.json'` to `APP_SHELL` and bump `CACHE_NAME` (`sw.js:1, 10-11`).

No JavaScript changes. The two shipped packs are structurally identical — **274 keys each, no key present in one and missing from the other** (verified by walking both trees).

Known limits — Steps 4 and 5 touch a Python file and the service worker, so "zero code changes" is true of the *app*, not of the repo. The Assamese pack is machine-authored and has not been reviewed by a native speaker (`README.md:111-112`). Numerals and dates come from `toLocaleDateString()` (`js/core/format.js:42`), which follows the device locale rather than the pack.

Claim discipline

What it does — Keeps forbidden vocabulary out of every user-visible string.

Where it lives — In three places. The rule is stated at the top of the two modules that could most easily break it:

```
// Claim discipline: everything computed here describes A
// Nothing infers a cognitive state, and no output may be
```

— `js/core/analytics.js:5-6`

```
// Claim discipline: this predicts SUCCESS IN AN ACTIVITY
// not a measure of the person, and it infers no cognitiv
```

— `js/core/model.js:12-13`

And the disclaimer appears on all three caregiver leaf screens: `content/en.json:640` (Overview), `:696` (Engagement), `:743` (Memories & Settings) — "This is not a diagnostic tool — consult a professional for concerns."

How it works — Because no string is hardcoded in JavaScript, the entire user-visible surface is two JSON files that can be read in one sitting. The activities are called "exercises" or "activities" throughout the packs; the word "game" appears in no user-visible string.

Known limits — This is enforced by discipline and review, not by a test. A lint rule over `content/*.json` for the forbidden list would take twenty lines and does not exist.

---

4. Data model

Five collections, all owned by `js/core/store.js` (`js/core/store.js:19`), each stored under its own key in the IndexedDB `kv` object store and mirrored to `localStorage` under `smriti:<key>`.

`reminders` — `Array<Reminder>`

Field	Type	Notes
`id`	string	`rem-<base36 time>-<random>` (`js/core/store.js:184-186`)
`type`	string	`medicine` | `hydration` | `activity` | `appointment` (`content/en.json:647-652`)
`time`	string	`"HH:MM"`, 24-hour
`label`	string	free note, may be empty
`createdAt`	number	epoch ms
`lastDoneAt`	number | null	acknowledged-at; "done today" is same-calendar-day (`js/core/store.js:251-263`)

Written by `addReminder` / `updateReminder` / `deleteReminder` / `markReminderDone` (`js/core/store.js:209-293`).

`memories` — `Array<Memory>`

Field	Type	Notes

`id`	string	`mem-…`
`name`	string	spoken aloud on the Memories screen
`image`	string	**data URL**, JPEG at ≤640 px long edge, quality 0.75
`createdAt`	number	epoch ms

Owned by `js/core/store.js:301-333`; produced by `js/screens/caregiver/memories-settings.js:21-46`. Two or more named memories also become Exercise 1 rounds (`js/screens/patient/exercises.js:330-351`).

`engagement` — `{ exercises: Record<id, ExerciseRecord>, sessions: Session[] }`

`ExerciseRecord`, created lazily per exercise (`js/core/store.js:368-380`):

Field	Type	Notes
`difficulty`	1–3	the level the **next** round will be played at
`correctStreak`, `wrongStreak`	number	rules-engine state
`lastChange`	`'up'` | `'down'` | `'repeat'` | null	describes the level the patient will actually meet (`js/core/store.js:437-442`)
`model`	`{ weights: number[4], trained: number }`	the learner (`js/core/model.js:45-47`)
`steering`	boolean	whether the learner is past its warm-up

`predicted`	number	its current expected success at the chosen level, 2 dp
`rounds`	`Round[]`	capped at 60 (`js/core/store.js:21, 426-428`)
`totalRounds`, `totalCorrect`	number	uncapped counters

`Round` (`js/core/store.js:417-425`): `{ ts, correct, promptId, attempts, difficulty, expected, change }`. `expected` is the model's pre-answer prediction, and is what the dashed overlay on the Engagement sparkline plots (`js/core/analytics.js:50-62`).

`Session` (`js/core/store.js:449-462`): `{ ts, startedAt, completed }`, capped at 30.

`settings` — object

`{ language, baselineDifficulty, sampleData, speechInput }` (`js/core/store.js:29-30`). `baselineDifficulty` is clamped to 1–3 on read and on every write (`js/core/store.js:131, 356-364`); setting it also resets every exercise's level and streaks while keeping history (`js/core/store.js:343-354`).

`activity` — object

`{ lastActiveAt, reminderLog }` (`js/core/store.js:31`). `reminderLog` is `{ reminderId, ts }[]`, capped at 200 (`js/core/store.js:286-290`).

Ownership summary

Module	Owns

`js/core/store.js`	all five collections, both backends, the write queue
`js/core/adaptive.js`	`difficulty`, `correctStreak`, `wrongStreak`, `lastChange` shape
`js/core/model.js`	the `model` sub-object and `proposeLevel`'s outputs
`js/core/analytics.js`	derived views only; owns nothing, stores nothing
`js/core/demo-data.js`	the seeded first-run payload

---

5. What is deliberately not built

Each of these is roadmap. None is stubbed, half-built or implied to exist.

Not built	One-line reason
Accounts, login, passwords	Adds a barrier in front of the exact user least able to clear it, and would be authentication theatre on a single shared device. The role toggle is a UX gate, stated as one (`README.md:104-106`).
Encryption at rest	Needs a key, and a key needs either a password (see above) or a hardware store a web page cannot reach. Encrypting with a key stored beside the data would be a claim, not a protection.
Cloud sync	Directly contradicts the offline-first constraint and would create a patient-data processor with an obligation the prototype cannot meet. Backup/restore to a file is the honest substitute (`js/screens/caregiver/memories-settings.js:247-249`).

Server-side or downloaded ML	A model file is a network dependency; a cloud call is a network dependency. Both break airplane-mode operation, which is a hard requirement.
Cognitive-condition inference	Would turn an engagement tool into a diagnostic claim, which is exactly the line the project refuses to cross (`js/core/model.js:12-13`).
Multi-patient health-worker workflow	Needs accounts and sync, both of which are excluded above.
Background/ OS-level reminder alarms	Requires push infrastructure, i.e. a server (`js/screens/patient/reminder-card.js:6-8`).

---

6. Verification — what is actually checked

There is no test runner, because there is no build step. There are four executable checks, three of which run in the browser.

`tools/model-harness.py` — the learner, in Python

Reads every constant out of `model.js` and `adaptive.js` by regex so the mirror cannot silently drift (`tools/model-harness.py:32-53`), then runs 4 synthetic patients × 40 seeds × 400 rounds, comparing the learner against the rules engine alone and against the best single level available to that patient. It fails the run if any patient lands more than 5 points off the best reachable rate, or further from it than the rules manage alone, or if the level oscillates on more than 20 % of rounds (`tools/model-harness.py:293-301`). Current status: **PASS**, with the table reproduced in section 3. It also emits `tools/model-convergence.svg` for the deck.

Proves: the control loop converges, tracks a drifting patient, beats the rules on realised success rate, and does not thrash. *Does not prove:*

anything about real patients. The synthetic patients are Bernoulli-per-level; a real person is not.

`tools/model-test.html` — the shipped JavaScript against that Python

Runs a fixed 20-round outcome pattern through the real `js/core/model.js` in a browser and asserts the resulting **levels, weights to 1e-9, and prediction** match the values `tools/model-harness.py --golden` prints (`tools/model-test.html:36-38, 88-99`). I re-ran `--golden` against the current source: the three golden constants in the test file are **exact matches** for what the harness produces today, so the mirror has not drifted.

It also brute-forces the guardrail — 1,200 proposals against random 30-round histories, asserting no proposal ever lands more than one level from the rules' choice or outside 1–3 (`tools/model-test.html:105-117`) — and feeds `null`, `NaN` weights and a string where an array belongs to check that a corrupted stored model resets to the prior instead of breaking the screen (`tools/model-test.html:119-128`, backed by `js/core/model.js:49-56`).

`tools/demo.html` — render-every-screen, and a driven session

Two buttons. **Render every screen** instantiates all twelve screens into an off-screen stage and reports anything that throws (`tools/demo.html:136-185`), including the reminder edit form pre-filling from a real saved record (`tools/demo.html:166-180`). **Play a full exercises session** drives a real session for up to 150 seconds and asserts: the intro auto-advances with no tap; a wrong tap produces feedback, reveals the correct answer and locks every option; **repeated taps during the pause do not skip the round**; all four later exercises are reachable; and the session returns to Patient Home on its own (`tools/demo.html:193-262`).

Proves: every screen executes, and the session state machine completes 1→2→3→4→5→praise. *Does not prove:* that anything looks

right, sounds right, or is legible. There are no visual or audio assertions.

`tools/store-test.html` — persistence

Roughly 30 assertions over reminders, memories, settings, the adaptive rules and engagement, including the two that matter: data survives dropping the in-memory cache and re-reading from disk (`store.reloadFromDisk()`), and data written by the *previous page load* survives a full reload — a marker planted in one run is checked at the start of the next, which is what makes it a real persistence test rather than a cache test. It reports which backend is live, IndexedDB or the localStorage fallback.

`tools/listen-test.html` — speech matching

18 cases over `matchSpoken()`, covering exact matches, casing, punctuation, sentence answers, one-letter errors and shouting — and, more importantly, the cases that must resolve to **nothing**: an unrelated word, a filler, an empty transcript, noise, "I do not know", a question, a two-letter fragment, and two equally confident alternatives. I ported the matcher to Python and ran all 18: **all 18 agree with the expected outcomes**, including every "resolves to nothing" case.

`tools/voice-test.html` — voice handover (device-only)

Six checks that an interrupted recording does not say itself again, that two recordings never overlap, that `stopSpeaking()` leaves actual silence, that a cancelled line never reports completion, that a completed line does, and that a line with no recording falls through to the device engine (`tools/voice-test.html:113-159`). It depends on real audio timing, so it must be run on the demo phone.

What remains untested

- **Every caregiver mutation path.** Adding a reminder, adding a

photo, deleting either, changing the language, toggling difficulty, export and restore are all exercised only by hand.
- **The service worker.** Nothing tests install, activation, cache eviction, or airplane-mode behaviour. That is a manual checklist item (`README.md:113–115`).
- **The four uncovered exercises' wrong-answer paths.** `tools/demo.html` asserts the reveal-and-advance behaviour for Exercise 1 only; the retry loops in 2–5 are driven but not asserted on.
- **Anything visual.** No screenshot, layout or contrast testing.
- **Assamese.** The pack is machine-authored and unreviewed by a native speaker.
- **Real devices.** Nothing here has been measured on a 1 GB Android phone.

---

7. Critical questions for the technical round

Twenty-eight questions, aimed at the joints. Each has the honest answer and the line to open.

Q1. You call this AI. Isn't it just if-statements?

Both are in there, and they do different jobs. `js/core/adaptive.js` is 54 lines of if-statements — three right in a row moves up, two wrong eases back — and it is deliberately that simple because it is the auditable floor a caregiver can be told about in one sentence. `js/core/model.js` is a logistic-regression classifier trained on-device by stochastic gradient descent from that patient's own rounds; it predicts success at each level and picks the one nearest a 75 % target. So: rules for the guarantee, a learner for the fit. Neither is a cloud call and neither is an LLM. **Proof:** `js/core/adaptive.js:26–54` and `js/core/model.js:100–149`.

Q2. You said no ML. But `model.js` is a trained classifier. Which is it?

The classifier is real, and calling it machine learning is correct. The project's standing rule forbids three specific things — an LLM, a cloud dependency, and a downloaded model file — and the code violates none of them: it starts from a four-number prior, trains only on this device, and needs no network. Where the documentation is wrong is `README.md:75-76`, which still says "there is no ML"; that line predates the learner and should read "no cloud ML, no LLM, no pretrained model." The user-facing copy has been correct throughout — the caregiver screen calls it "On-device learning" and explains what it does (`content/en.json:704-710`). **Proof:** `js/core/model.js:1-17` for the design note, `README.md:73-76` for the stale line.

Q3. Why logistic regression rather than just a bigger rules table?

A rules table can only encode what someone thought of in advance, and it cannot represent *this* patient — every user gets the same table. Logistic regression gives a calibrated probability per level from four features, which is exactly the quantity needed to answer "which level lands nearest 75 %". It is also inspectable: four weights, printable, and the harness prints them (`bias=0.963, level=-1.121, recentAccuracy=0.043, recentRetries=-0.125` on the current run). And measurably, it wins: 75.6 % vs 62.9 % realised success for the steady patient, 70.7 % vs 56.2 % for the struggling one. **Proof:** `js/core/model.js:75-95`, `tools/model-harness.py:262-301`.

Q4. Why not a neural network?

Three reasons, in order of weight. It must train from scratch on a 1 GB phone from a few dozen examples — a net would overfit or need a pretrained initialisation, and a pretrained initialisation is a file to download, which breaks offline-first. It must be auditable by a clinician, and four named weights are; a hidden layer is not. And a net cannot beat the ceiling here: with three levels and four features the decision surface is one-dimensional in practice. **Proof:** `js/core/model.js:6-10`, `js/core/model.js:38-43`.

Q5. Cold start. What does the learner do on round one, and why is that safe?

Nothing. It predicts and it trains, but `proposeLevel` returns the rules' level unchanged until it has seen six rounds, and the caregiver screen

says so in plain words: "Watching for now — it starts adjusting after *n* more rounds." Before that the rules engine is entirely in charge, which is the same behaviour the app had before the learner existed. **Proof:** `js/core/model.js:29-32, 125-132`; UI at `js/screens/caregiver/engagement.js:136-143` and `content/en.json:706`.

Q6. How do you know the learner helps at all? What is the evidence?

`tools/model-harness.py` runs four synthetic patients with known true per-level success rates, 400 rounds × 40 seeds, and compares the learner against the rules engine alone and against the best any single level could have done. The learner beats the rules by 12.7, 2.2, 14.5 and 5.6 points of realised success, lands within 5 points of the best reachable rate in all four cases, and changes level on only 2–8 % of rounds. Calibration error is 0.046–0.077. The honest caveat: these are simulated patients drawn from a Bernoulli-per-level model. This shows the control loop is sound. It is not clinical evidence and we do not present it as any. **Proof:** `tools/model-harness.py:215-220` for the patients, `:293-301` for the pass criteria.

Q7. How do I know the JavaScript you ship does what your Python harness validated?

`tools/model-test.html` runs a fixed 20-outcome pattern through the real `js/core/model.js` in the browser and asserts identical levels, identical weights to nine decimal places, and an identical prediction against constants generated by `model-harness.py --golden`. I re-ran the generator against current source and the three constants in the test file match exactly. **Proof:** `tools/model-test.html:36-38, 88-99`; `tools/model-harness.py:231-251`.

Q8. What stops the learner from doing something stupid — jumping a patient straight to the hardest level after a lucky streak?

Three things, all constants at the top of one file. It only considers `[rules - 1, rules + 1]`, so it can never move more than one step from where the auditable rules engine landed. A candidate must be at least 0.03 closer to the target to win, and ties fall to the *easier* level. And it does not act at all before six rounds. The guardrail is brute-forced:

1,200 proposals against random histories, zero breaches. **Proof:** `js/core/model.js:120-150`; `tools/model-test.html:105-117`.

Q9. What does the model actually see? Does it know anything about the person?

Four numbers: a bias, the level (−1/0/+1), first-try accuracy over the last 8 rounds, and retry count over the same window. No age, no name, no diagnosis, no timing, no session data, nothing typed by the caregiver. Every feature is computable before the round is answered, which is enforced by passing the history *as it stood before* this answer. **Proof:** `js/core/model.js:72-87`; the ordering at `js/core/store.js:404-410`.

Q10. You claim offline. What exactly breaks in airplane mode?

One thing: speech recognition. The browser's recogniser is cloud-backed on Android Chrome — it streams audio to a server, and there is no offline speech recognition available to a web page. We do not hide it: the code says so, the caregiver setting says so in the patient's caregiver's own words, and a network error retires the microphone for the session rather than leaving a dead button on screen. Everything else — every screen, all five exercises, both content packs, every image, all 191 recorded English lines, the learner, the store — is precached and works with the radio off. Second-order: Assamese audio is not precached, so a device that switches language before ever being online falls back to live synthesis for that language. **Proof:** `js/core/listen.js:11-16` and `:77-85`; `content/en.json:733`; `sw.js:86-90`.

Q11. Privacy. Where does patient data live, and what leaves the device?

Photos, names, reminders, and practice history live in IndexedDB on that one phone, mirrored to localStorage. There is no server, no account, no analytics, no telemetry, and no runtime CDN request — that last one is a hard constraint, because a CDN request would also break airplane mode. The single exception is speech input: when the patient presses the microphone, the browser's own recogniser sends that audio to its service. We did not build that pipe and cannot inspect it, so we disclose it, we let the caregiver switch it off in one tap, and the app is

fully usable with it off. **Proof:** `js/core/store.js:1-5`; `js/core/listen.js:11-14`; `js/screens/caregiver/memories-settings.js:222-245`; `content/en.json:733`.

Q12. So a patient's face and name are sitting unencrypted on a phone?

Yes, and we would rather say that than claim otherwise. Encryption at rest needs a key; a key needs a password, and a password in front of a person with dementia is a barrier they cannot clear — or it needs a hardware keystore a web page cannot reach. Storing a key beside the data would be a claim, not a protection. The mitigations we did build are real: nothing leaves the phone, the data is one device's worth rather than a database of patients, and the backup is a file the caregiver moves by hand rather than an account they can lose. For a deployed product the answer is a native wrapper with OS-level keystore and device encryption, and that is on the roadmap, not in the demo. **Proof:** `js/screens/caregiver/memories-settings.js:250-272`; `README.md:98-106`.

Q13. Why no login, and what stops the wrong person opening the caregiver side?

Nothing stops them, and we call it a UX gate rather than authentication for exactly that reason. The corner requires a deliberate two-second hold on a dim 44 px target, which is enough that a disoriented patient cannot stumble into a screen of their own performance data — that is the threat it is designed for. It is not designed for an adversary, because on a single-device, no-account, no-server app there is no adversary model that a password would actually address: anyone holding the phone already has the data. **Proof:** `js/screens/patient/home.js:35-45`; `README.md:104-106`.

Q14. What happens on a 1 GB Android phone with no TTS voice installed?

Better than it used to. 191 English lines are shipped as recordings and play through an `<audio>` element with no engine involved, so the intro, all the feedback, the praise, Exercise 2's and Exercise 5's prompts and every fixed line still speak. Eight lines are assembled at runtime from a name, a time or an object — the reminder card, Today's next-reminder

sentence, "Tap the gamosa", "morning — let us put these in order", "Which one is Ama?" — and those need a live engine. With no engine, those are silent and shown as text only. That is a real gap and I would fix the voice-status readout first, because it currently reports the device as having a voice whenever recordings exist, which slightly overstates the case. **Proof:** `js/core/voice.js:125-141` for the two-source path; `js/core/voice.js:106-114` for the overstatement; `tools/build-voice.py:68-72` for why templated lines are skipped.

Q15. What if the phone's speech engine never calls you back? Does the app freeze?

No. Nothing in the app is ever on the speech path. Every screen that advances on narration pairs the voice callback with its own ceiling timer and fires whichever comes first, exactly once. For a recorded line the ceiling is widened by the clip's real measured duration, so a recording is never cut off by a guess. **Proof:** `js/screens/patient/exercises.js:91-102` and `:297-300`; `js/core/voice.js:1-3`.

Q16. Regulatory: what makes this not a medical device?

It makes no claim to diagnose, treat, prevent, mitigate or monitor a disease. It reports what happened in the app — rounds completed, first-try answers, days practised, the level the activities are set to — and nothing else. There is no score presented to the patient, no threshold that means anything clinically, and no output phrased as a statement about the person; the two modules that could most easily cross that line carry the rule as a comment at the top. Every caregiver screen carries "This is not a diagnostic tool — consult a professional for concerns." What would make it a device is a claim, and we have made none. **Proof:** `js/core/analytics.js:5-6`, `js/core/model.js:12-13`, `content/en.json:640, 696, 743`.

Q17. Your alerts say "has been harder lately". Isn't that a clinical inference in disguise?

It is descriptive and thresholded, and the threshold is arbitrary and stated: six recent rounds with fewer than 40 % first-try correct. The string names the activity, not the person — "Remembering has been harder lately — fewer first-try answers." It does not say the patient has declined, and the surrounding copy tells the caregiver to consult a

professional. If a reviewer wanted that alert removed we would remove it; it is one entry in a JSON file. **Proof:** `js/core/analytics.js:102-104, 131-148`; `content/en.json:770`.

Q18. Accessibility: a user with tremor. What happens on a mis-tap?

Minimum touch target is 64 px on every control, well above the 48 px Android guideline. Nothing depends on a swipe, a pinch, a long-press or a multi-touch gesture in normal patient use — the only long-press in the app is the caregiver gate, and it is deliberately hard. In four of the five exercises a wrong tap does not end the round: it is marked, gently narrated, cleared and reopened, and input is held during the pause so a tremor cannot double-fire through it. In the fifth, every option locks on the first tap so a second tap does nothing at all. That last behaviour is asserted by the driven session test. **Proof:** `css/base.css:33, 88-89`; `js/screens/patient/exercises.js:534, 549-558` (retry path) and `:378-382` (lock path); `tools/demo.html:219-222`.

Q19. Accessibility: cataracts, or low vision.

Root font is 18 px against a browser default of 16. Body text on every patient screen is 1.6 rem — about 29 px — and the entry heading is 3.4 rem, about 61 px. Body ink on background measures **14.5:1** and the accent on white **7.2:1**, both comfortably past WCAG AAA for normal text. Focus is a 3 px solid outline with offset. The honest exception: the photo counter — "3 of 5" — uses the faintest ink token at 1 rem, which measures **3.03:1** and fails AA for body text. It is decorative rather than load-bearing, but it is a genuine defect and it is on the fix list. **Proof:** `css/base.css:14-16, 48, 104`; `css/patient.css:16, 24, 406`.

Q20. Accessibility: hearing loss. The app leads with voice.

Every spoken line is also on screen as text — voice leads, it never carries alone. The exercises can be completed end to end with the sound off; nothing waits on audio and nothing advances only on audio. Feedback is colour *and* text, and the feedback line is `aria-live="polite"` so a screen reader announces it. What is missing: there is no captioning of the *pacing*, so a deaf user on a self-advancing screen sees the text appear and disappear on a timer they did not set. **Proof:** `js/screens/patient/exercises.js:129-131,`

`193-196`; `js/core/voice.js:1-3`.

Q21. How is this different from a flashcard app?

Three things a flashcard app does not do. It leads with a recorded human-sounding voice on every screen and requires no reading. It adapts per activity, per patient, on-device, and shows the caregiver that adaptation as a chart of prediction against outcome rather than as a claim. And it is a whole-day companion — reminders, a Today screen, a named-photo browser that feeds the recall activity — rather than a drill. The activities are the visible part; the reminders and the caregiver record are what a family would actually keep it installed for. **Proof:** `js/screens/patient/reminder-card.js:1-8`; `js/screens/caregiver/engagement.js:102-120`; `js/screens/patient/exercises.js:330-351`.

Q22. Prove the adaptive part does anything on the demo device, right now.

Open Caregiver → Engagement. Each activity shows its current level, the last ten round outcomes as marks, a first-try accuracy line with the model's expectation drawn over it as a dashed overlay, and a level line under it. Then: the seeded demo leaves the first two activities one correct answer short of a level rise and capped below the maximum, so three correct answers in Remembering will move the level in front of you, the patient will hear "You're doing well — let's try a little harder", and the Engagement card will change. That is deliberate staging of the *starting state*, not of the rules — the rules are untouched. **Proof:** `js/core/demo-data.js:116-125`; `js/screens/patient/exercises.js:181-186`.

Q23. Why is there fake data on the device at all? Are you showing me an invented patient?

Yes, and the app says so on the screen that owns it. A cold install shows an empty state everywhere, which reads as a broken app rather than a new one, so first run seeds a fortnight of plausible history. It is flagged: Caregiver → Memories & Settings opens with a "Sample data" banner explaining that everything shown is an example, and one tap clears it and leaves a blank device. The seeded rounds train the learner exactly as real ones would, so the Engagement screen opens on a

model that has genuinely learned from that history rather than a blank one claiming otherwise. **Proof:** `js/app.js:28-32`; `js/screens/caregiver/memories-settings.js:312-327`; `js/core/demo-data.js:74-78`.

Q24. Scale: 100 patients, one health worker. What breaks first?

Everything, immediately, and by design. There is no account, no server and no sync, so there is no such thing as a second patient on one device or one patient across two devices — the only bridge is a JSON backup file moved by hand. A health-worker workflow needs identity, which needs accounts, which needs a server, which breaks the offline constraint the whole app is built on. That is a different product with a different threat model and a different regulatory posture, and building a fake version of it here would have been the dishonest choice. What exists today is the honest primitive it would be built on: a complete, exportable, single-device record. **Proof:** `js/screens/caregiver/memories-settings.js:247-272`; `README.md:98-102`.

Q25. Your reminders — will they wake the patient if the app is closed?

No, and we say so in the source, in the README and here. A web page cannot register an OS-level alarm without a push server, and a push server is a network dependency the app is built to do without. What we did build is a 30-second watcher so a reminder arrives *while the app is open* rather than only when Home is redrawn, and it deliberately does not interrupt an exercise in progress. The real fix is a thin native wrapper that owns the alarm and opens the PWA; that is roadmap. **Proof:** `js/screens/patient/reminder-card.js:3-8, 16-19, 60-74`.

Q26. How would you add a third language, and what would you have to change?

In the app, nothing. Drop `content/ne.json` next to the other packs, add one entry to `app.languages` in each pack so it appears in the caregiver's language select, and set `app.speechLang` to the ordered voice-fallback chain. No JavaScript changes: every string, image path and language tag already comes from the pack, and both shipped packs are structurally identical at 274 keys with no divergence. Two things outside the app do change: add the voice to `tools/build-voice.py`

if you want recordings, and add the pack to `APP_SHELL` and bump `CACHE_NAME` so it precaches. So "zero code changes" is true of the application; it is not true of the repository, and I would not claim otherwise. **Proof:** `js/core/content.js:8-17`; `content/as.json:8-13`; `js/screens/caregiver/memories-settings.js:145-163`; `tools/build-voice.py:45-48`; `sw.js:10-11`.

Q27. Why does Assamese use a Bengali voice? Isn't that a shortcut?

It is a deliberate choice with a linguistic reason. Assamese neural voices do not exist in the TTS service we build with, and Android phones almost never carry an Assamese system voice. Assamese is written in the Bengali-Assamese script, so a Bengali voice renders the graphemes far closer than any Latin-script voice would. So both layers use the same chain: the recording build renders Assamese with `bn-IN-TanishaaNeural`, and the live fallback list is `as-IN → as → bn-IN → bn`, which prefers a real Assamese voice if the device happens to have one. The caregiver screen reports which voice the device actually found, so nobody has to guess. The honest caveat is that a Bengali voice reading Assamese is approximately, not natively, correct — and the Assamese pack itself is machine-authored and has not been reviewed by a native speaker. **Proof:** `tools/build-voice.py:41-48`; `content/as.json:8-13`; `js/core/voice.js:34-38`; `js/screens/caregiver/memories-settings.js:166-198`; `README.md:111-112`.

Q28. What is the weakest part of this codebase, and what would you fix first?

In order:

1. **No background reminders.** The app can only remind you while it is open. This is the largest gap between what the product promises and what it does. Fix: a thin native wrapper owning the alarm. `js/screens/patient/reminder-card.js:6-8`.
2. **The caregiver side has no automated coverage at all.** Add a reminder, add a photo, restore a backup — all hand-tested. A judge who breaks something will break it there. `tools/`

`demo.html:136-149` renders those screens but asserts nothing about their mutations.

3. **The voice-status readout overstates.** It reports a device as having a voice whenever recordings exist, but eight templated lines still need a live engine — so a phone with no engine gets a green dot and a silent "Tap the gamosa." `js/core/voice.js:106-114`.
4. **Route params live outside the URL.** No screen is addressable or restorable; reload and hardware-back replay stale params. Invisible on the patient rails, but it is a latent bug. `js/core/router.js:5, 12`.
5. **Claim discipline is enforced by review, not by a check.** A twenty-line lint over `content/*.json` for the forbidden vocabulary would make it structural. It does not exist.

Beyond those: the localStorage mirror swallows quota failures silently (`js/core/store.js:93-99`); the manifest's maskable icon is not precached (`manifest.json:12` vs `sw.js:38-43`); `clearAll()` leaves a wiped device reporting "Last active: Just now" (`js/core/store.js:484-486`); and speech input is wired into Exercise 1 only, not all five (`js/screens/patient/exercises.js:403`).

Vocabulary note: this document uses "engagement", "activity performance", "session history" and "practice activities" throughout, and calls the activities "exercises" or "activities". It contains no claim that the app measures, detects, diagnoses or improves cognition, because it does none of those things.